

第3章 传输层



学习目标:

- 掌握传输层服务原理:
 - 多路复用, 多路分解
 - 可靠数据传输
 - 流量控制
 - 拥塞控制
- 学习因特网传输层协议:
 - UDP: 无连接传输
 - TCP: 面向连接的可靠传输

3-1

第3章 传输层



3.1 传输层服务

3.2 多路复用与分解

3.3 无连接传输: UDP

3.4 可靠数据传输原理

3.5 面向连接的传输: TCP

- 报文段结构
- 可靠数据传输
- 流量控制
- TCP连接管理

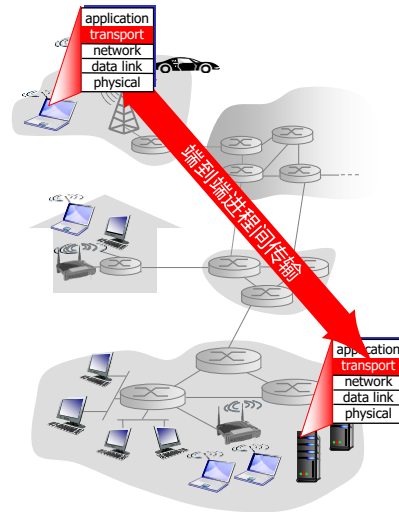
3.6 拥塞控制原理

3.7 TCP 拥塞控制

3-2

传输层服务与协议

- 在不同主机进程间提供端到端通信服务
- 传输层协议运行于端系统
 - 发送方: 将应用层报文切分成报文段, 交给网络层
 - 接收方: 将从网络层收到的报文段重组为报文, 交给应用层
- 传输层协议:
 - 因特网: TCP和UDP



3-3

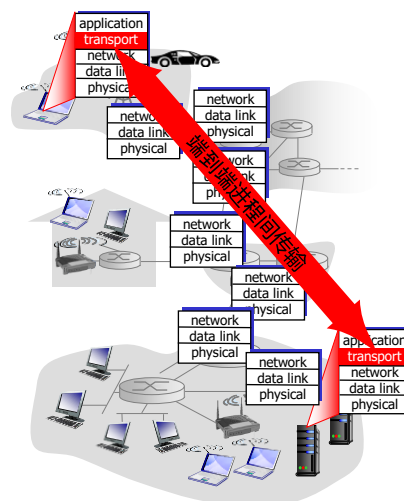
传输层 vs. 网络层

- 网络层: 源主机与目的主机间通信
- 传输层: 源主机进程与目的主机进程间通信
 - 基于网络层服务, 并进一步实现进程通信

3-4

因特网传输层协议

- 可靠，按序传输 (TCP)
 - 建立、拆除连接
 - 流量控制
 - 拥塞控制
- 不可靠，不保证按序到达 (UDP)
 - 基于网络层的“尽力而为” IP协议，只增加了进程通信必要功能
- 都不提供：
 - 时延保证
 - 带宽保证



3-5

第3章 传输层



3.1 传输层服务

3.2 多路复用与分解

3.3 无连接传输: UDP

3.4 可靠数据传输原理

3.5 面向连接的传输: TCP

- 报文段结构
- 可靠数据传输
- 流量控制
- TCP连接管理

3.6 拥塞控制原理

3.7 TCP 拥塞控制

3-6

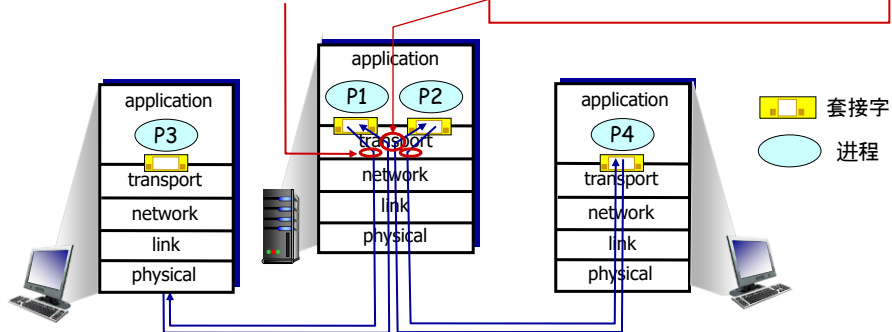
传输层的多路复用与多路分解

发送方的多路复用：

从多个套接字接收应用数据块，加上首部，交给网络层

接收方的多路分解：

从网络层收到数据块，根据首部信息交给不同的套接字



3-7

多路分解的方法

从网络层收到IP数据报

- 每个数据报含有源IP和目的IP地址
- 每个数据报含有一个传输层报文段
- 每个报文段含有源端口和目的端口号



TCP/UDP 报文段

- 根据目的IP地址和端口号，送往对应的套接字

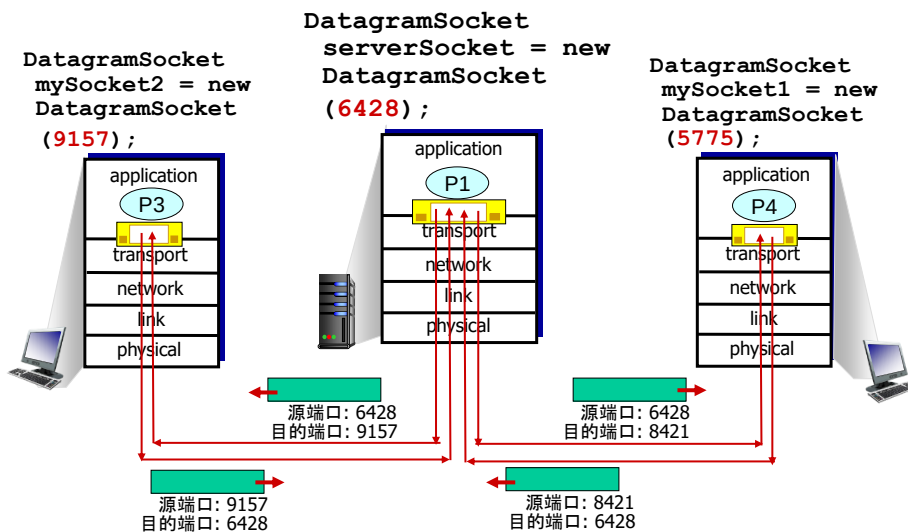
3-8

无连接的多路分解

- 回顾一下，
套接字有一个本地端口号：
`DatagramSocket mySocket1 = new DatagramSocket(12534);`
 - 向UDP套接字发送的数据报，需要携带：
 - 目的IP地址
 - 目的端口号
 - 目的主机收到UDP数据报：
 - 检查数据报的目的端口号
 - 将数据报发送到此端口对应的套接字
- 具有相同目的IP地址和端口号的数据报，被送往相同的UDP套接字

3-9

无连接的多路分解：举例



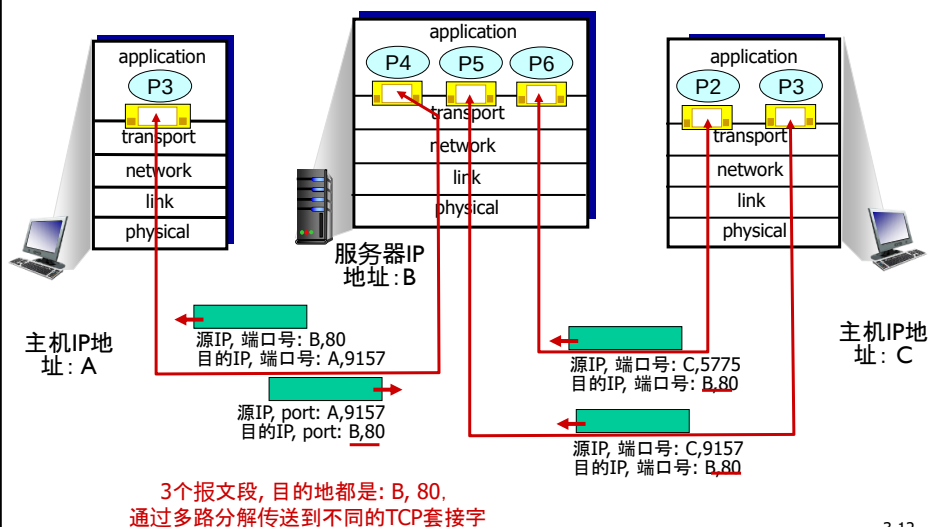
3-10

面向连接的多路分解

- TCP套接字由4元组确定:
 - 源IP地址
 - 源端口号
 - 目的IP地址
 - 目的端口号
- 多路分解: 接收方使用4元组将报文段发给不同的目的TCP套接字
- 服务器进程通常可有多
个TCP套接字:
 - 每个套接字含唯一的4元组
- 例如: Web服务器针对
每一个客户机有不同的
TCP套接字

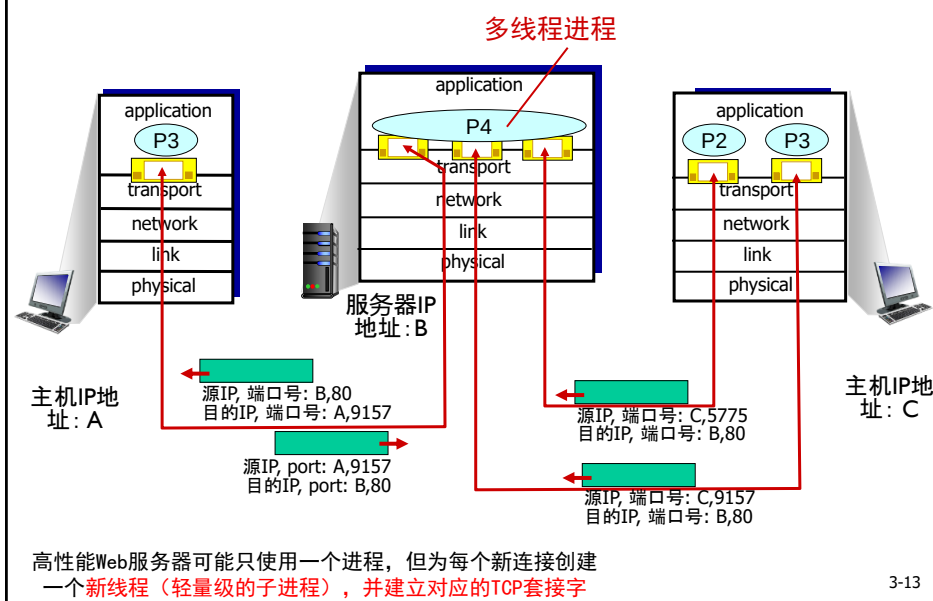
3-11

面向连接的多路分解：举例



3-12

面向连接的多路分解：举例



3-13

第3章 传输层



3.1 传输层服务

3.2 多路复用与分解

3.3 无连接传输: UDP

3.4 可靠数据传输原理

3.5 面向连接的传输: TCP

- 报文段结构
- 可靠数据传输
- 流量控制
- TCP连接管理

3.6 拥塞控制原理

3.7 TCP 拥塞控制

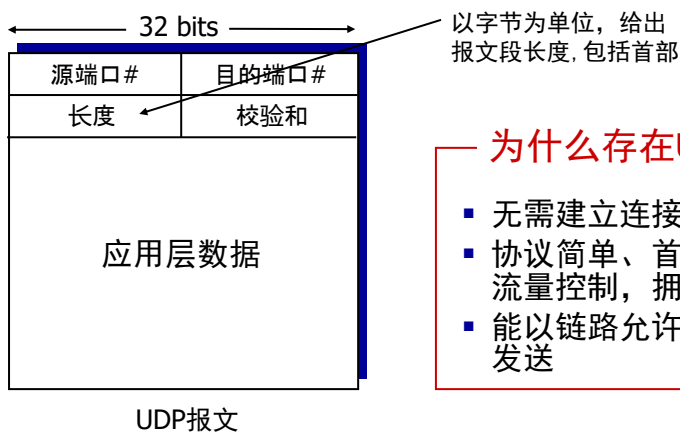
3-14

UDP: 用户数据报协议 [RFC 768]

- 不提供不必要服务的传输层协议
- “尽力而为”，UDP 数据报可能:
 - 出错
 - 丢失
 - 失序
- 无连接:
 - 无握手过程
 - 每个数据报含目的IP地址和端口号，独立传送
- 应用:
 - 流媒体传输 (容忍少量丢包，带宽敏感)
 - DNS
 - SNMP
- 基于UDP的可靠传输:
 - 在应用层增加编号，确认和重传等

3-15

UDP报文结构



为什么存在UDP?

- 无需建立连接
- 协议简单、首部简单：无流量控制，拥塞控制
- 能以链路允许的最大速率发送

3-16

UDP校验和

作用:端到端差错检测

发送方:

- 将报文段 (包括首部) 切分成16比特的块
- 求和 (高位进位加到低位), 再取反, 得到校验和
- 放入校验和字段, 发送

接收方:

- 对接收到的信息 (包括校验和) 求和
 - 全 "1" : 数据无错;
 - 其中有 "0" : 数据出错
- 对接收到的信息 (不包括校验和) 求和, 核对计算的和并取反是否等于校验和字段的值
 - 相等: 数据无错
 - 不等: 数据出错

3-17

校验和: 例子

有三个16 比特的字:

	0 1 1 0 0 1 1 0 0 1 1 0 0 0 0 0
	0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1
	1 0 0 0 1 1 1 1 0 0 0 0 1 1 0 0
回卷	① 0 1 0 0 1 0 1 0 1 1 0 0 0 0 0 1
和	0 1 0 0 1 0 1 0 1 1 0 0 0 0 1 0

取反, 校验和为: 1 0 1 1 0 1 0 1 0 0 1 1 1 1 0 1

无差错, 和为: 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1

注意: 作加法时, 最高位的进位要回加到结果中。

3-18

第3章 传输层



3.1 传输层服务

3.2 多路复用与分解

3.3 无连接传输: UDP

3.4 可靠数据传输原理

3.5 面向连接的传输: TCP

- 报文段结构
- 可靠数据传输
- 流量控制
- TCP连接管理

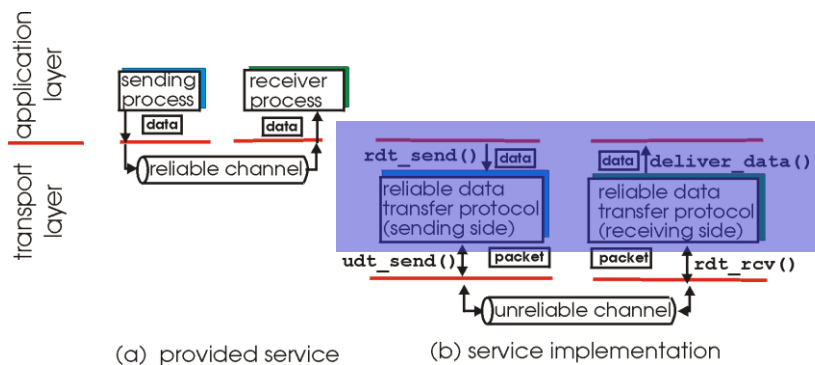
3.6 拥塞控制原理

3.7 TCP 拥塞控制

3-19

可靠数据传输原理

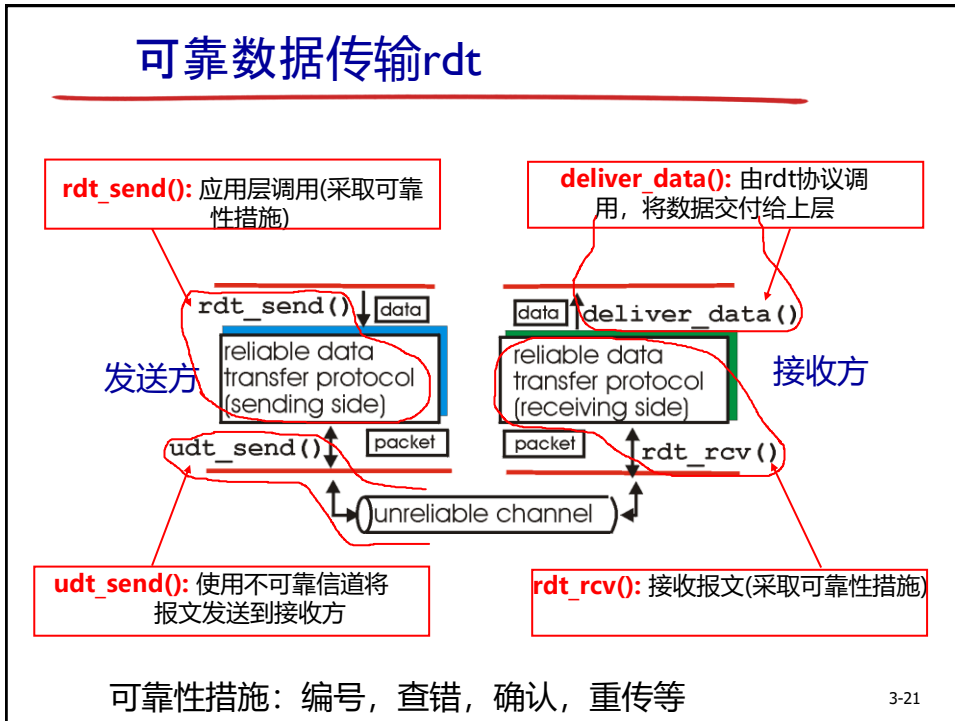
- 可靠数据传输 (reliable data transfer, rdt) 是应用层、传输层、链路层的重要问题
 - 也是网络最重要的10个问题之一!



- 信道的特点决定rdt协议的复杂度

3-20

可靠数据传输rdt

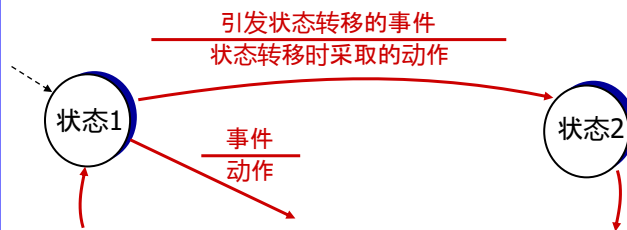


可靠数据传输rdt

接下来:

- 由易到难, 构建rdt的发送方和接收方
- 只考虑单向数据传输
 - 控制信息可以双向
- 使用**有限状态机** (FSM: finite state machine) 建模

- 箭头:** 表示协议从一个状态变迁到另一个状态
- 横线上方:** 表示引起变迁的事件
- 横线下方:** 表示事件发生时所采取的动作
- Λ:** 表示没有事件或未采取动作
- 虚线:** 表示FSM的初始状态



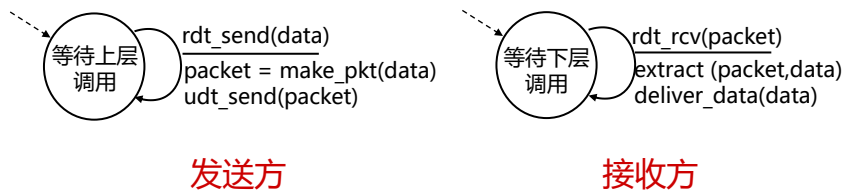
rdt1.0: 可靠信道上的可靠传输

- 假设条件:

- 底层信道是可靠的

- 无比特错误
 - 无丢包

- 有限状态机:



3-23

rdt2.0: 有比特差错的信道

- 假设条件:

- 可能产生比特错，但不会丢包，且保证按序

- 报文比特可能翻转

- 检错: 校验和

- 如何纠错? :

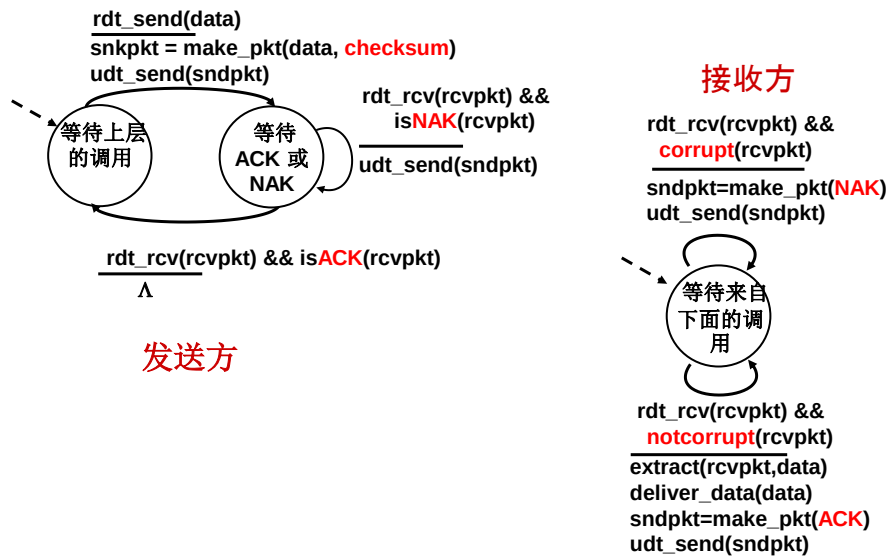
- 肯定确认 (ACK): 接收方告诉发送方, 报文正确收到
 - 否定确认 (NAK): 接收方告诉发送方, 报文有错
 - 发送方收到NAK后重发报文

- 在rdt1.0基础上, rdt2.0的新措施:

- 差错检测
 - 确认: ACK, NAK, 接收方到发送方
 - 重传

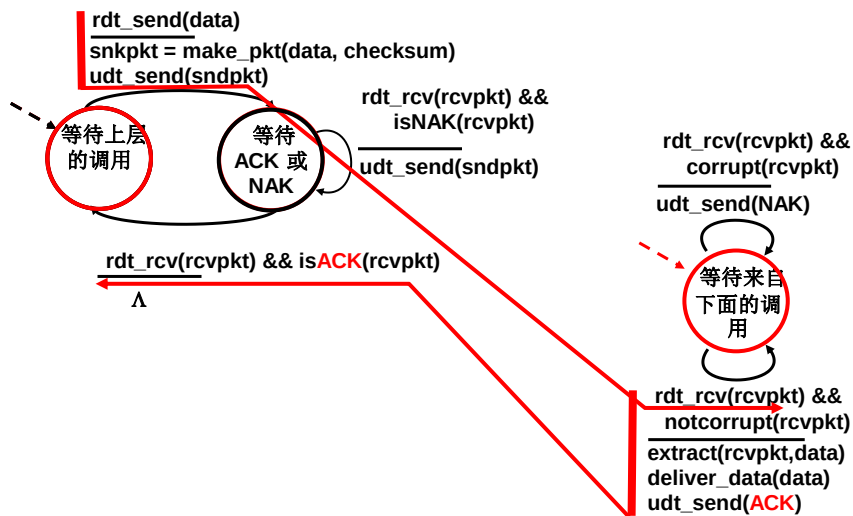
3-24

rdt2.0: FSM表示



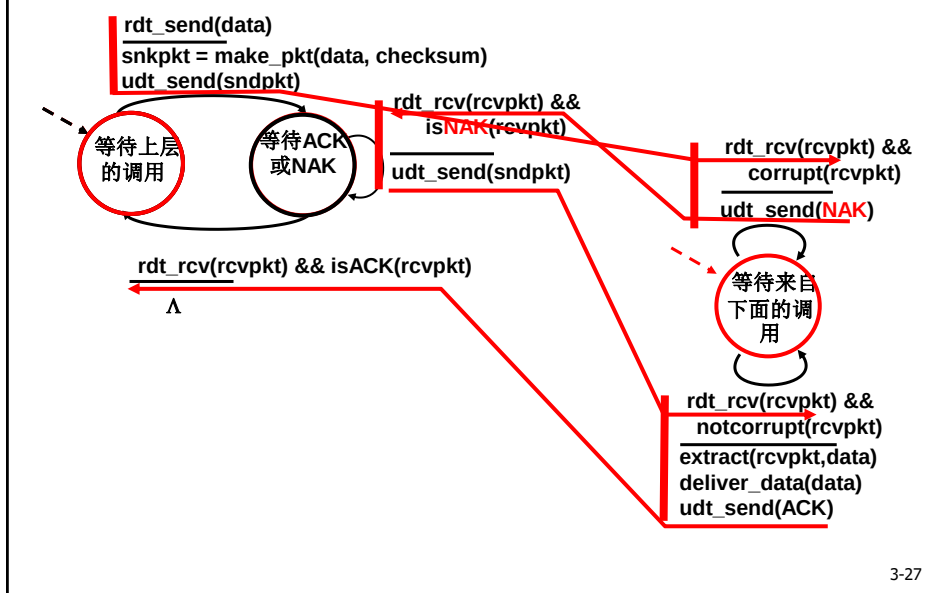
3-25

rdt2.0: 分组无差错的操作



3-26

rdt2.0:分组有差错的操作



3-27

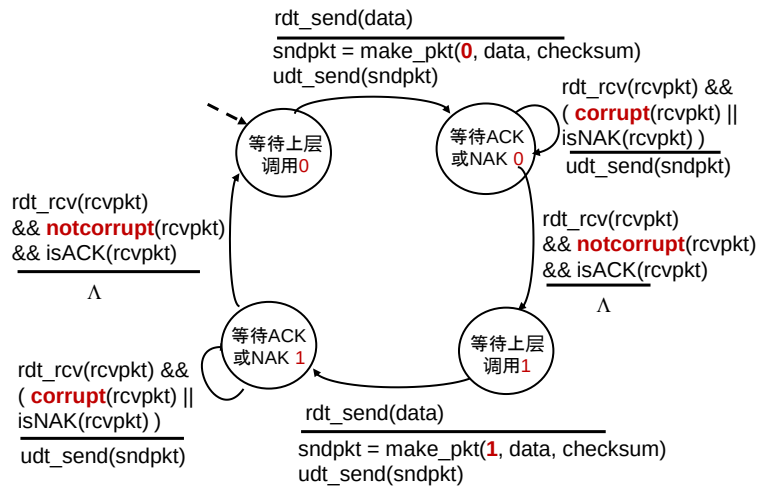
rdt2.0的缺陷

问题：如果ACK/NAK出错？ 解决方法：

- 发送方不知道接收方是否正
确接收前一数据报文！
- 只重传：导致冗余报文
- 接收方ACK/NAK增加校验和
- 如果ACK/NAK出错，发送方
重传前一数据报文
- 发送方数据报文增加序号
- 接收方根据序号判断和丢弃
冗余报文

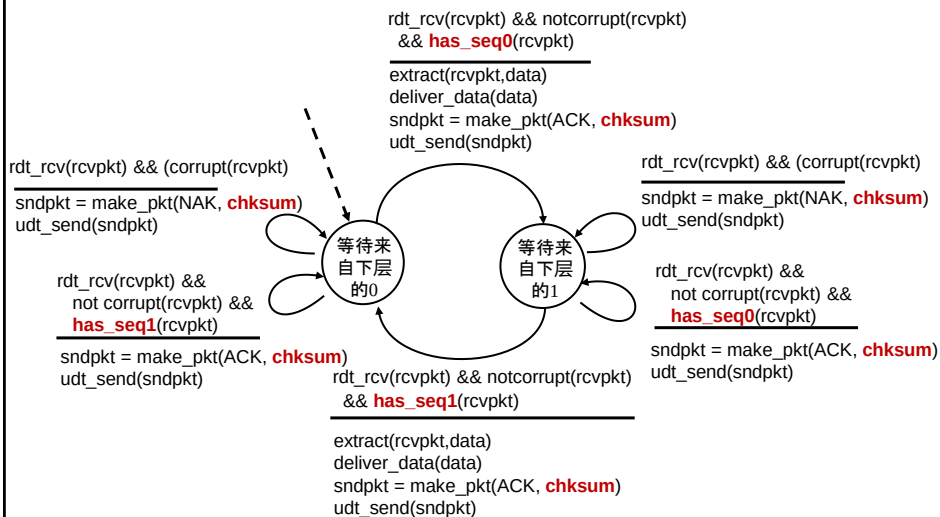
3-28

rdt2.1: 发送方



3-29

rdt2.1: 接收方



3-30

rdt2.1 (应对ACK/NAK出错): 讨论

发送方:

- 数据报文添加序号
- 两个序号(0,1) 就足够了. Why?
- 检查收到的ACK/NAK是否出错

接收方:

- ACK/NAK增加校验和
- 检查收到的分组是否冗余

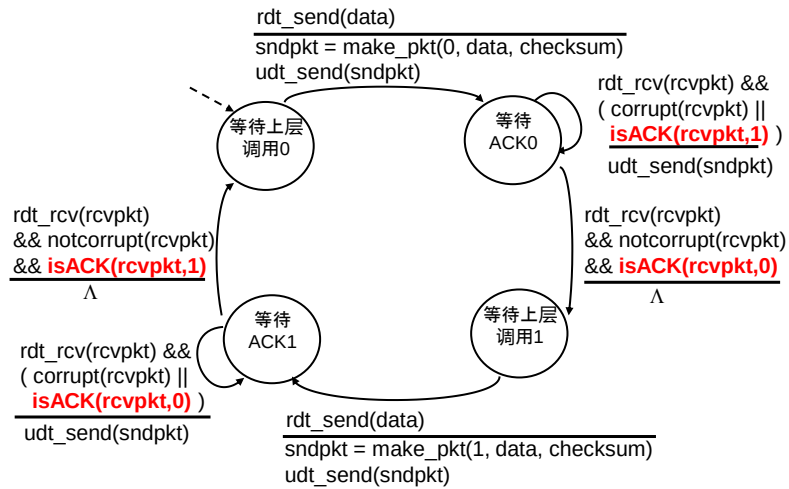
3-31

rdt2.2 (无NAK的协议)

- 与rdt2.1相同, 但只使用ACK
- 接收方每次返回最后正确收到分组的ACK
- 发送方如收到冗余ACK, 重传当前分组 (已发送过的)

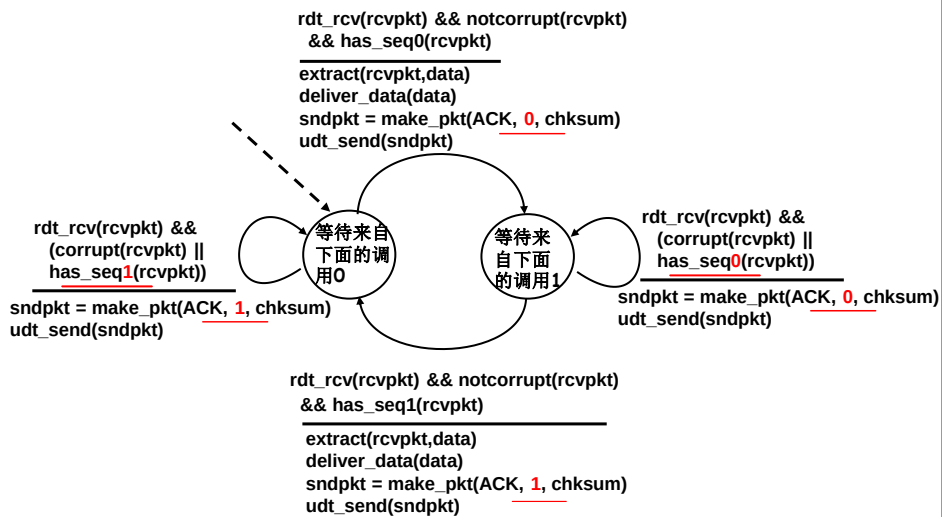
3-32

rdt2.2: 发送方



3-33

rdt2.2: 接收方



3-34

rdt3.0: 出错且丢包的信道

新假设条件: 底层信道

解决方案: 发送方发送报

可能出错且丢包

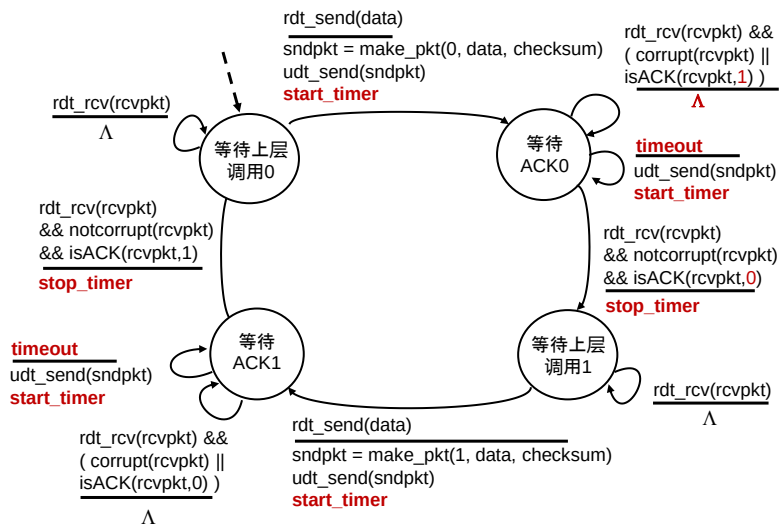
文后, 启用**定时器**

- 使用校验和、ACK、重传、序号还不够
- 数据报文可能**丢失**
- **ACK**可能**丢失**

- 如果在一定时间内未收到ACK, 则重传
- 如果数据报文或ACK只是晚到了 (未丢包):
 - 过早超时
 - 重传导致冗余, 但序号可以处理冗余

3-35

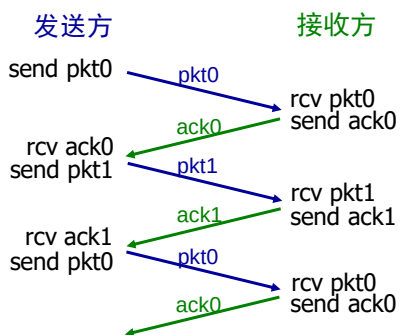
rdt3.0发送方



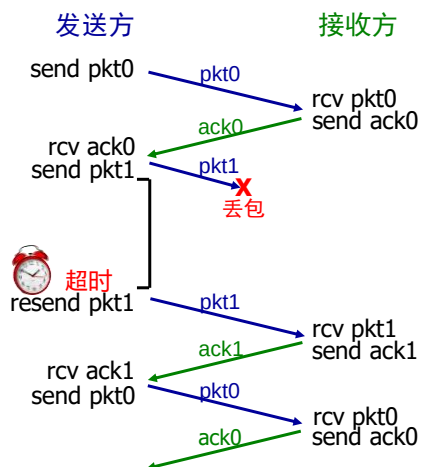
接收方和2.2相同

3-36

rdt3.0运行情况



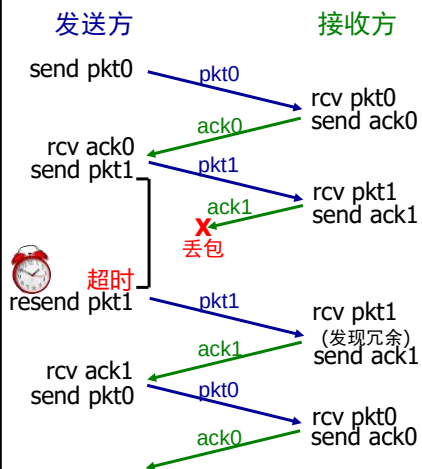
(a) 无丢包



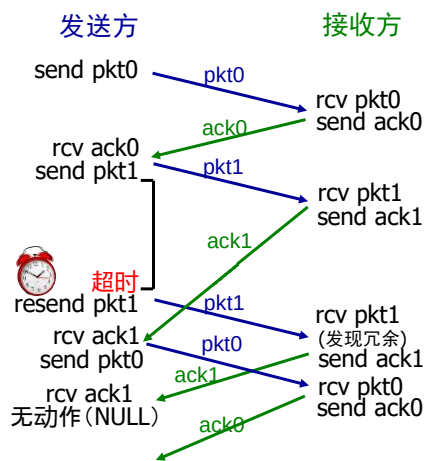
(b) 有丢包

3-37

rdt3.0运行情况



(c) ACK丢失



(d) 过早超时

3-38

rdt3.0的性能

- rdt3.0正确，但性能差（停等协议）
- 发送方(信道)利用率 U_{sender} :

发送方将比特发送到信道的时间

发送时间

e.g.: 1Gbps的信道, 15ms的单向传播时延, 8000 bit的报文:

$$D_{\text{trans}} = \frac{L}{R} = \frac{8000 \text{ bits}}{10^9 \text{ bits/sec}} = 8 \text{ microsecs}$$

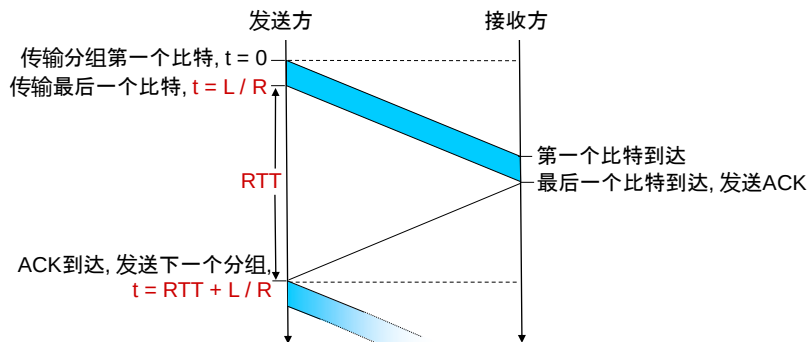
停等协议

发送方发送一个分组，停下来等待对方的响应； rdt2.x, rdt3.0

传输时延: L/R

发送方从发送分组开始，到收到对方ACK分组所需的时间：
 $RTT + L/R$

rdt3.0的性能



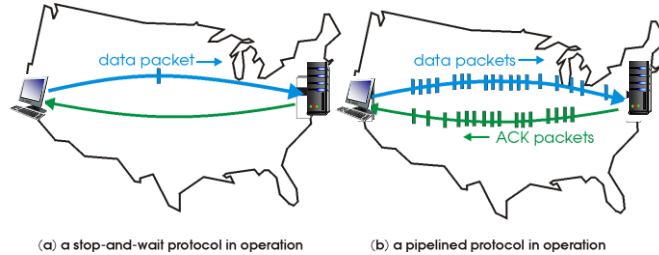
$$U_{\text{sender}} = \frac{L/R}{RTT + L/R} = \frac{.008}{30.008} = 0.00027$$

- 协议限制了资源利用率！

流水线协议 (pipeline)

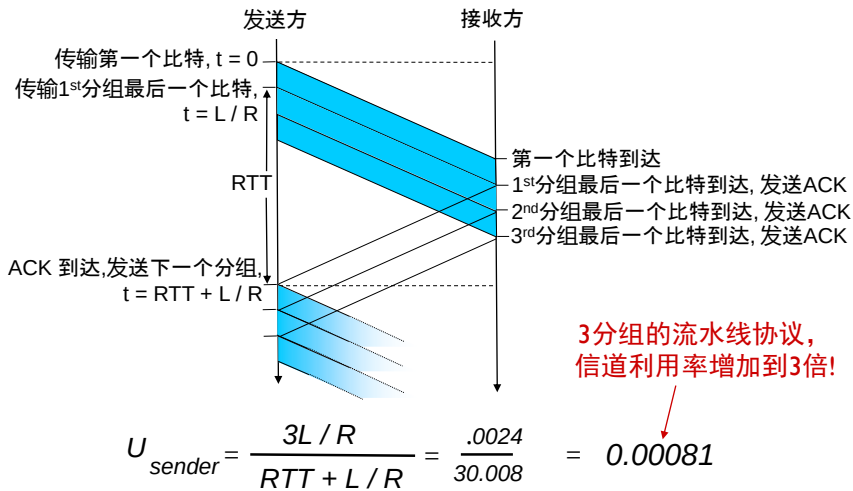
流水线：发送方可连续发送多个报文，无需等待确认

- 双方缓存报文（已发送未确认，已接收但失序）
- 增加序号范围



3-41

流水线协议：增加信道利用率



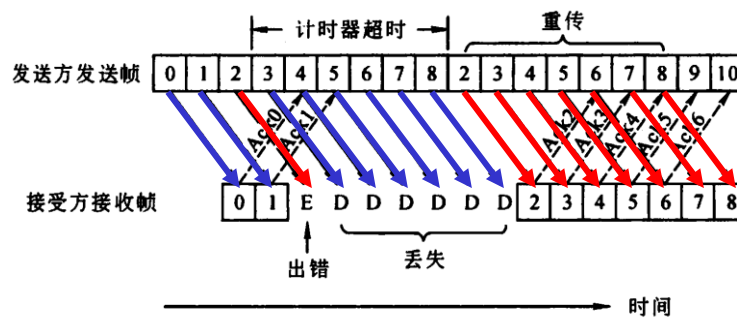
3-42

流水线协议

- 回退N步 (Go-Back-N, GBN协议)
- 选择性重传 (Selective Repeat, SR协议)

3-43

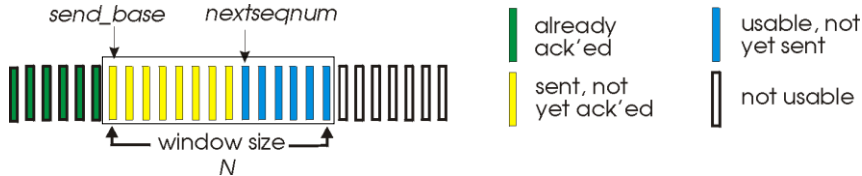
回退N步：工作示意图



3-44

回退N步: 发送方

- 使用k比特的序号字段
- `base(send_base)`指针: 窗口起始位置; `next(nextseqnum)`指针: 下一个可发送分组



已确认 已发未确认 可发送 不能发送

- $[0, \text{base}-1]$: 已经发送并确认过的分组。
- $[\text{base}, \text{next}-1]$: 已经发送但未被确认分组
- $[\text{next}, \text{base}+N-1]$: 用于将立即发送的分组
- $\geq \text{base}+N$: 不能使用, 直到当前流水线中未被确认的分组得到确认(序号为base的分组)

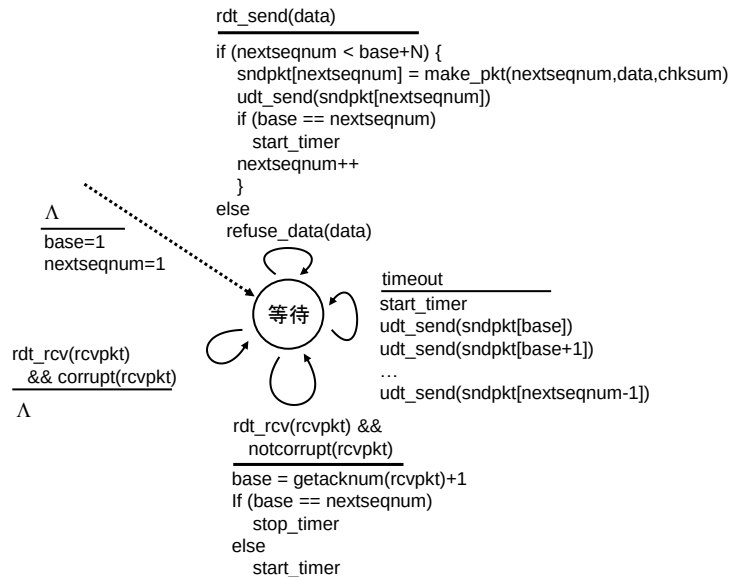
3-45

回退N步: 发送方

- 发送窗口 $W_T = N$, 即序号范围 $[\text{base}, \text{base}+N-1]$, 包括已发送未被确认的分组、以及可发送的分组
- 随着协议的运行, 发送的分组陆续被确认, 发送窗口在序号空间内向前滑动
- GBN协议常被称为滑动窗口协议(sliding-window protocol)
- 窗口已满 (窗口中全都是已发未确认分组) 条件:
 $\text{next} = \text{base} + N$
- 窗口未满 (准备发送 > 0) 条件:
 $\text{next} < \text{base} + N$
- 窗口为空条件: $\text{next} == \text{base}$
- 累积确认, 操作指针和定时器
 - 移动窗口
 - 关闭或重启定时器
 - 可能冗余确认
- 只对最老未确认分组使用定时器
- 超时: 重传窗口中所有未确认分组

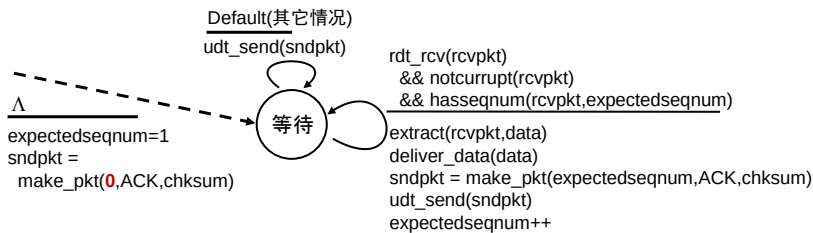
3-46

回退N步: 发送方



3-47

回退N步: 接收方

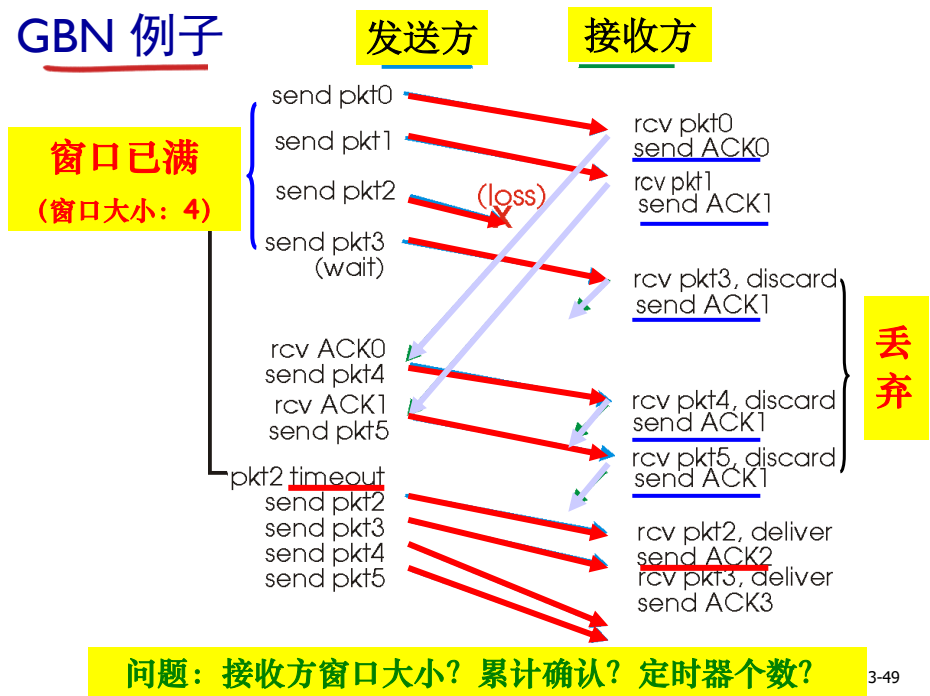


累积确认:

- 可能产生冗余确认
- 接收方只需记住**expectedseqnum**指针 (下一个期待分组)
- 收到失序分组:
 - 丢弃 (不缓存): **GBN接收方无缓存** (接收方窗口大小为1) !
而发送方有 (发送方窗口大小为N)
 - 重发

3-48

GBN 例子



GBN 例子: 发送窗口最大尺寸

若序号位数 k 位, GBN 协议, 发送窗口最大尺寸是

$$W_T = 2^k - 1, \text{ 而不是 } 2^k.$$

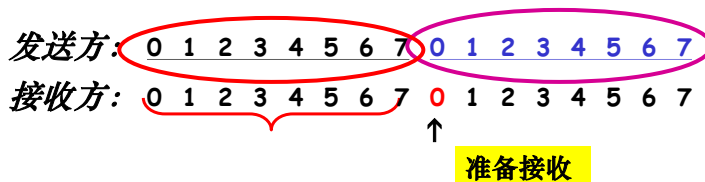
举例说明:

设序号 3 位, 则序号空间为 $0\# \sim 7\#$ (共 8 个), 其发送窗口最大尺寸是 $W_T = 7$, 而不是 8 。

GBN 例子：发送窗口最大尺寸

(1) $W_T = 8$

- ① 发方发送0# ~ 7#共8分组，停止等待应答；
 - ② 收方全部正确收到，分别发应答，窗口序号变为0#（准备收0#分组）；
- ✓ 应答全部到达：发方接着发送8个新分组0# ~ 7#，协议正确进行；
 - ✓ 应答全部丢失：发方超时，重新发送8个旧分组0# ~ 7#，收方出现重复分组接收，协议错误。

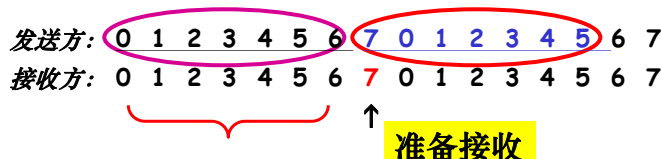


3-51

GBN 例子：发送窗口最大尺寸

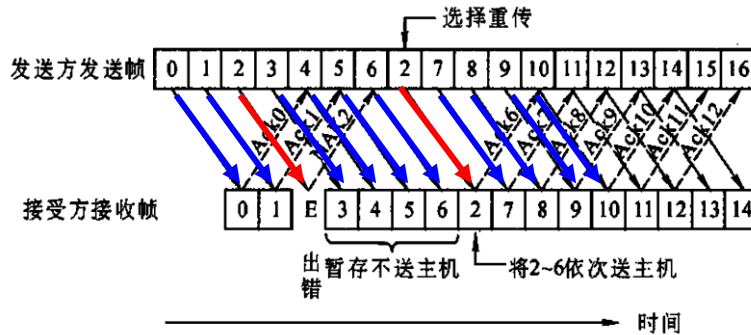
(2) $W_T = 7$

- ① 发方发送0# ~ 6#共7分组，停止等待应答；
 - ② 收方全部正确收到，分别发应答，窗口序号变为7#（准备收7#分组）；
- ✓ 应答全部到达：发方接着发送7个新分组7#、0# ~ 5#，协议正确进行；
 - ✓ 应答全部丢失：发方超时，重新发送7个旧分组0# ~ 6#，与收方准备接收的分组序号不相同，拒绝接收，丢弃。



3-52

选择重传（SR）：工作示意图



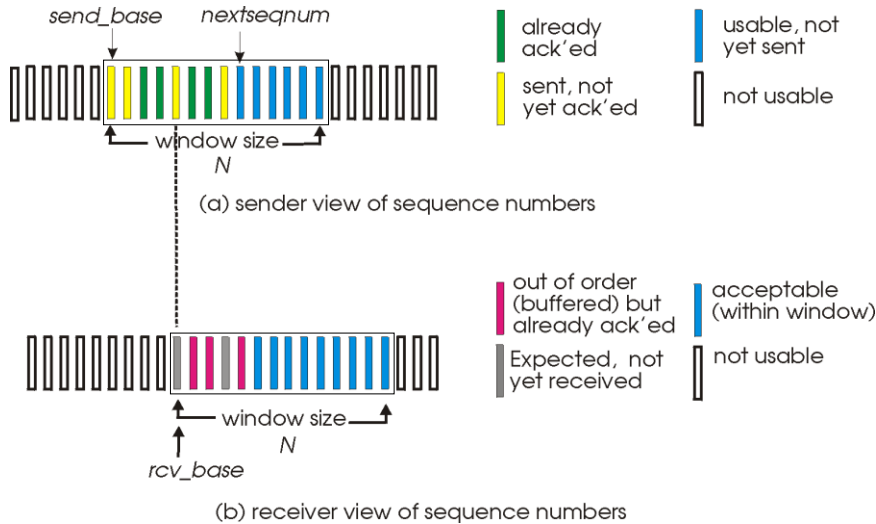
3-53

选择重传

- 接收方对每个正确接收分组发送确认（包括失序分组）
 - 接收方缓存失序分组，实现最终按序交付
- 发送方只重传超时分组
 - 对每个未确认分组使用定时器
- 双方都使用大小为N的缓存
 - 发送窗口(N): base指向第一个未确认分组；收到确认号=base，窗口滑动，使base指向第一个未确认分组
 - 接收窗口(N): base指向第一个未接收；收到分组编号=base，窗口滑动，使base指向第一个未接收分组

3-54

选择重传：发送方、接收方窗口



3-55

选择重传

发送方

从上层收到数据:

- 如果 *next* 指针位于窗口内，发送分组

超时(*n*):

- 重发对应分组，重启定时器

收到 ACK(*n*) 位于

[*sendbase*, *sendbase* + *N*):

- 将对应分组标记为已确认
- 如果 *n* = 窗口基序号，移动窗口，使 *base* 指向下一个未确认分组

接收方

收到分组 *n* 位于 [*rcvbase*, *rcvbase* + *N* - 1]

- 发送 ACK(*n*)
- 失序分组: 缓存
- 按序分组: 交付 (一并交付后续按序分组), 移动窗口到下一个未接收分组

收到分组 *n* 位于 [*rcvbase* - *N*, *rcvbase* - 1]

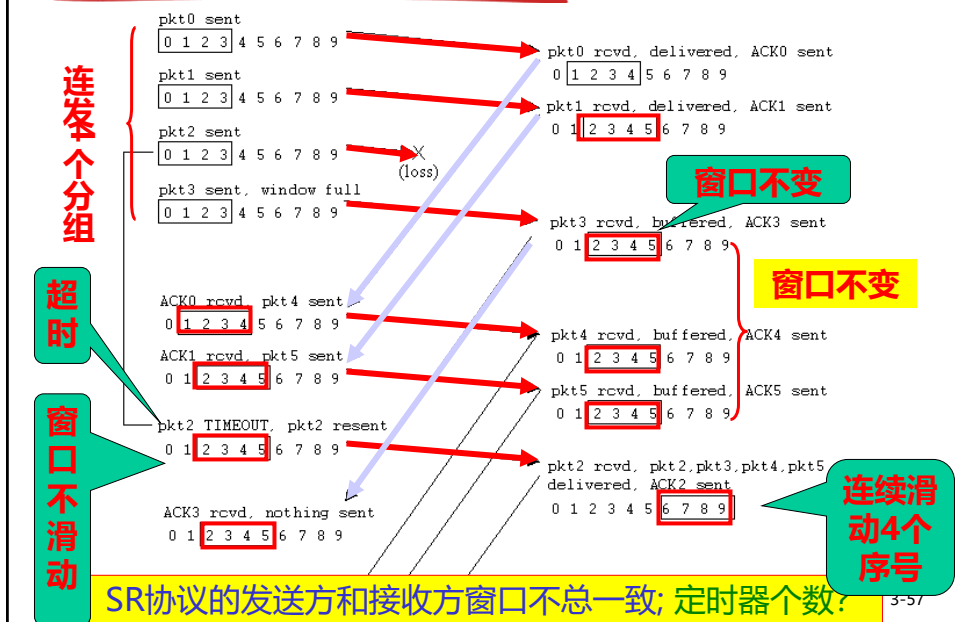
- ACK(*n*); 以免由于 ACK 丢失，导致发送窗口无法移动

其它情况:

- 忽略

3-56

选择重传例子



选择重传：窗口最大尺寸

若序号位数 k 位，SR协议，发送窗口和接收窗口尺寸最大是 2^{k-1} ，不是 $2^k - 1$ 。即序号空间一半。

反例说明：

设序号 2 位，则序号空间为 $0\# \sim 3\#$ ，发送窗口和接收窗口尺寸最大是 2 ，不是 3 。

选择重传：窗口最大尺寸

(1) $W_T = W_R = 3$

- ①发方发送0#~2#共3分组，停止等待应答；
- ②收方全部正确收到，发应答，窗口滑动，允许接收3#、0#、1#分组；
 - ✓ 应答全部到达：发方发送3个新分组3#、0#、1#，协议正确进行；
 - ✓ 应答全部丢失：发方超时，重新发送0#~2#共3旧分组，与准备接收序号重叠，出现重复分组接收，协议错误。

发送方: 0 1 2 3 0 1 2 3
接收方: 0 1 2 3 0 1 2 3
 ↑
 准备接收

3-59

选择重传：窗口最大尺寸

(2) $W_T = W_R = 2$

- ①发方发送0#~1#共2分组，停止等待应答；
- ②收方全部正确收到，发应答，窗口滑动，允许接收2#~3#分组；
 - ✓ 应答全部到达：发方发送2个新分组2#~3#，协议正确进行；
 - ✓ 应答全部丢失：发方超时，重新发送2个旧分组0#~1#，序号不在接收窗口内，拒绝接收，丢弃。

发送方: 0 1 2 3 0 1 2 3
接收方: 0 1 2 3 0 1 2 3
 ↑
 准备接收

3-60

流水线协议总结

回退N步:

- 发送方收到确认前，最多可连续发送N个分组
- 接收方发送**累积确认**
 - 表明当前分组及之前分组已正确接收
- 发送方只对最老未确认分组使用定时器
 - 如超时，重传所有未确认分组

选择重传:

- 发送方收到确认前，最多可连续发送N个分组
- 接收方对每个收到分组分别发送确认
- 发送方对所有未确认分组使用定时器
 - 如超时，只重传对应的未确认分组

3-61

第3章 传输层



3.1 传输层服务

3.2 多路复用与分解

3.3 无连接传输: UDP

3.4 可靠数据传输原理

3.5 面向连接的传输: TCP

- 报文段结构
- 可靠数据传输
- 流量控制
- TCP连接管理

3.6 拥塞控制原理

3.7 TCP 拥塞控制

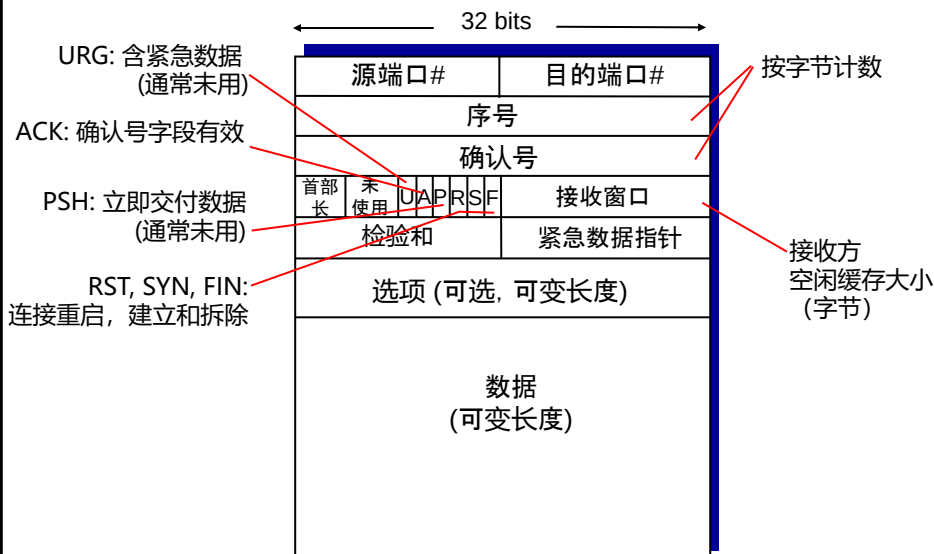
3-62

TCP: 传输控制协议

- 点对点
- 可靠字节流传输
- 流水线协议:
 - 流量控制和拥塞控制决定发送方窗口大小
- 全双工传输:
 - MSS: 最大段大小（由本地网络链路层帧大小决定）
- 面向连接:
 - 三次握手建立连接
 - 四次握手拆除连接
- 流量控制:
 - 控制发送方发送速度，使接收方来得及接收
- 拥塞控制:
 - 控制发送方发送速度，使网络来得及转发

3-63

TCP报文段



3-64

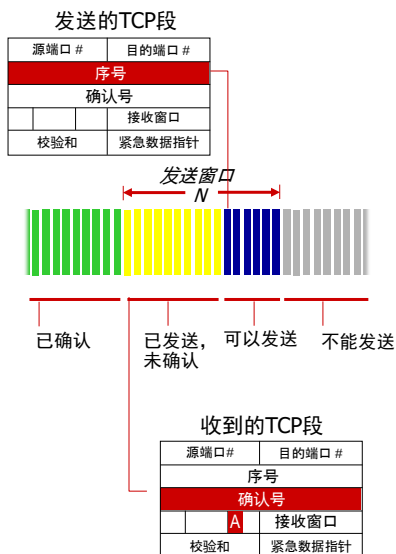
TCP 序号，确认号

序号:

- 数据首字节的编号

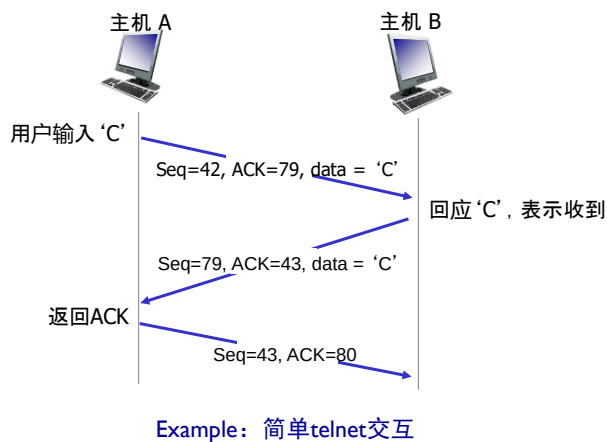
确认号:

- 希望对方发送的下一个起始字节编号
- 累积确认



3-65

TCP 序号、确认号



3-66

TCP超时值与往返时延

Q: TCP 超时值?

- 太短: 过早超时, 重传
- 太长: 对丢包反应慢
- 比RTT长一些
 - 但RTT动态变化

Q: 如何估计RTT?

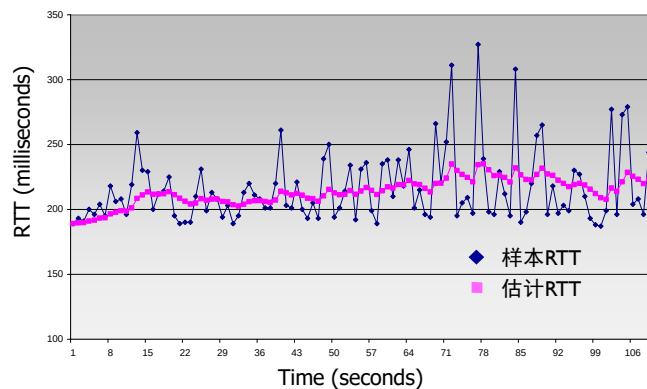
- 样本RTT: 测量发出报文到收到确认所需时间
 - 不考虑重传
 - 波动, 希望 “更平滑”

3-67

TCP 往返时延

$$\text{估计RTT} = (1 - \alpha) * \text{估计RTT} + \alpha * \text{样本RTT}$$

- 通常: $\alpha = 0.125$
- 过去样本的影响随时间指数衰减
- 更平滑



3-68

TCP 的超时值

- 超时值 = 估计RTT + 安全余量
- RTT变动大 -> RTT偏差大 -> 安全余量大

安全余量 = $4 * RTT\text{偏差}$

$RTT\text{偏差} = (1-\beta) * RTT\text{偏差} + \beta * |\text{样本RTT} - \text{估计RTT}|$

(通常, $\beta = 0.25$)

超时值 = 估计RTT + $4 * RTT\text{偏差}$



↑
“安全余量”

3-69

第3章 传输层



3.1 传输层服务

3.2 多路复用与分解

3.3 无连接传输: UDP

3.4 可靠数据传输原理

3.5 面向连接的传输: TCP

- 报文段结构
- 可靠数据传输
- 流量控制
- TCP连接管理

3.6 拥塞控制原理

3.7 TCP 拥塞控制

3-70

TCP 可靠数据传输

- 差错控制：TCP在不可靠IP之上提供rdt
 - “管道”传输字节流
 - 累积确认
 - 单个重传定时器
 - 通常缓存失序报文
 - GBN和SR结合
- 重传的原因：
 - 定时器超时
 - 收到冗余确认
- 流量控制
- 拥塞控制
- 面向连接

3-71

第3章 传输层



3.1 传输层服务

3.2 多路复用与分解

3.3 无连接传输: UDP

3.4 可靠数据传输原理

3.5 面向连接的传输: TCP

- 报文段结构
- 可靠数据传输
- 流量控制
- TCP连接管理

3.6 拥塞控制原理

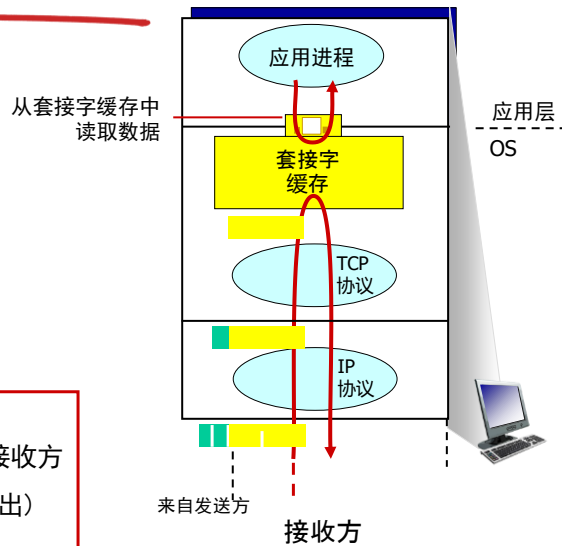
3.7 TCP 拥塞控制

3-72

TCP流量控制

流量控制

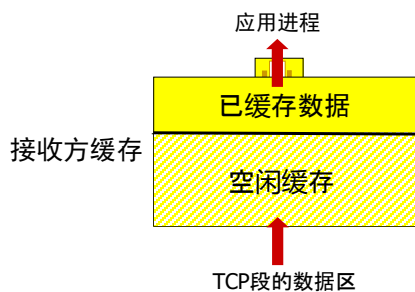
控制发送方速度，使得接收方来得及接收(避免缓存溢出)



3-73

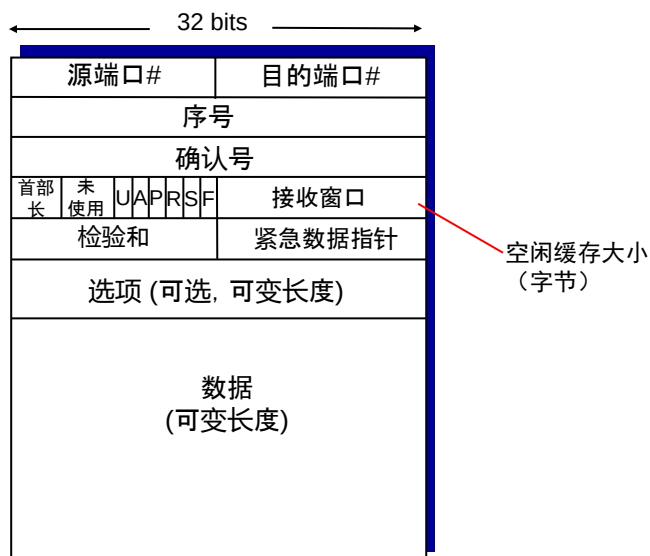
TCP流量控制

- 接收方通过**接收窗口**字段将空闲缓存大小告知发送方
 - 默认值为4096字节
- 发送方当前可发送字节数 < 接收窗口
- 避免接收缓存溢出



3-74

TCP报文段



3-75

第3章 传输层



3.1 传输层服务

3.2 多路复用与分解

3.3 无连接传输: UDP

3.4 可靠数据传输原理

3.5 面向连接的传输: TCP

- 报文段结构
- 可靠数据传输
- 流量控制
- TCP连接管理

3.6 拥塞控制原理

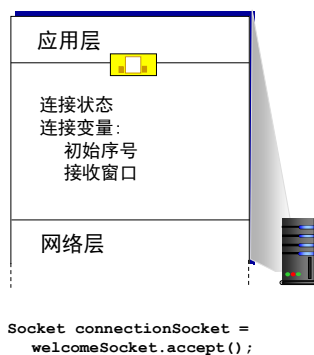
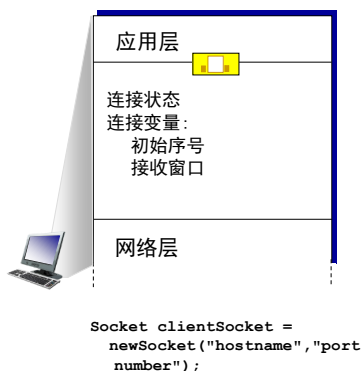
3.7 TCP 拥塞控制

3-76

TCP连接管理

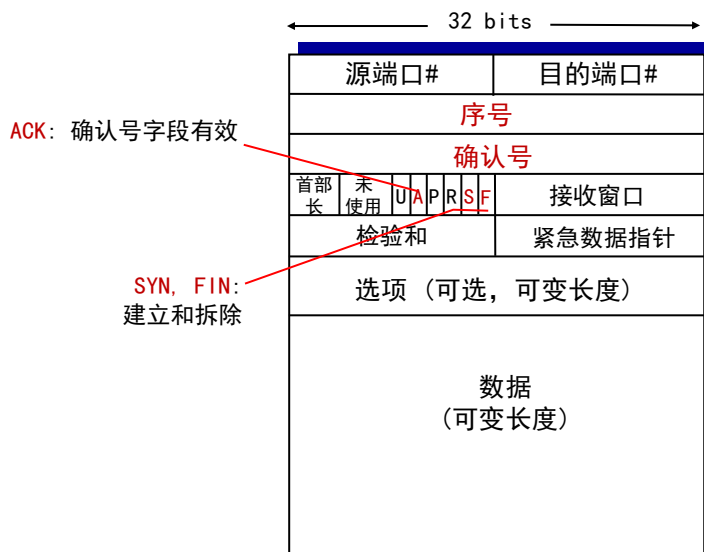
在传送数据前，双方进行握手：

- 双方同意建立连接
- 协商参数



3-77

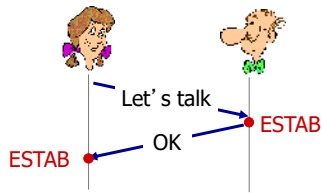
TCP连接相关字段



3-78

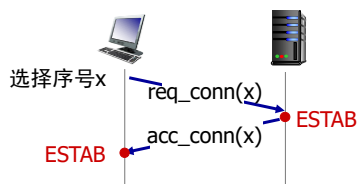
建立连接

两次握手:



Q: 两次握手可用于网络?

A: No

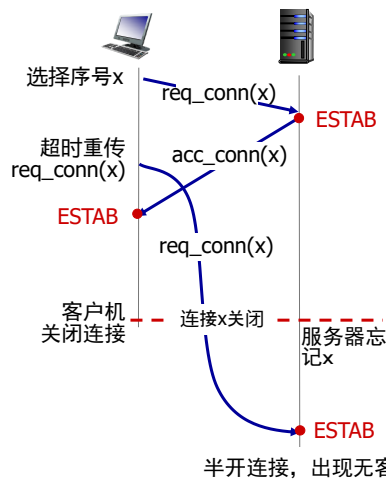


不同之处:

- 时延变动
- 报文乱序
- 差错与丢包导致重传
- 看不到对方

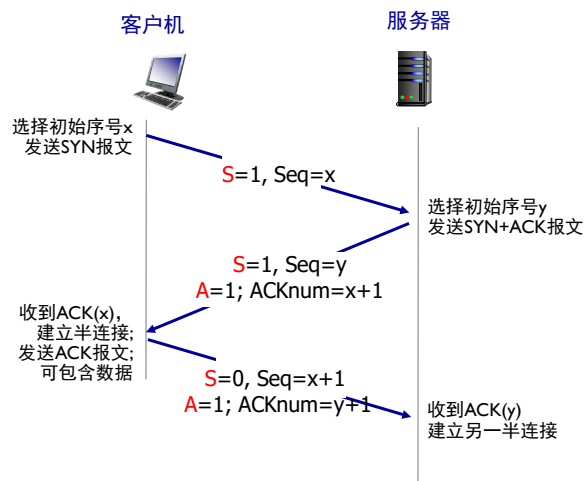
3-79

两次握手的问题



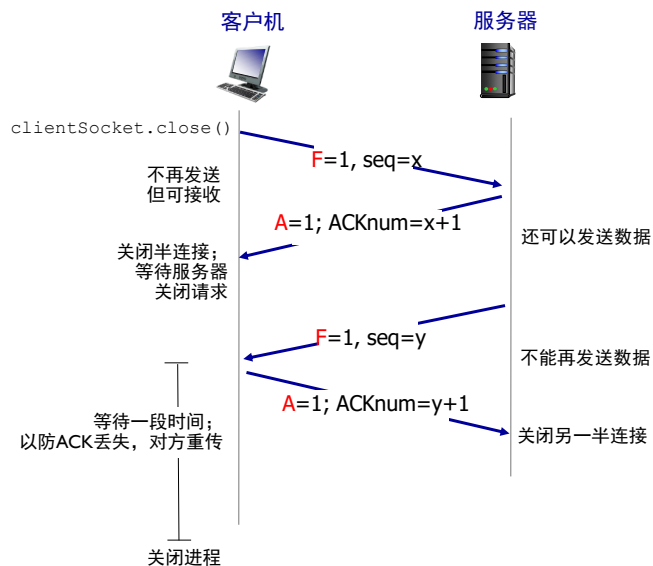
3-80

三次握手建立TCP连接



3-81

四次握手关闭TCP连接



3-82

第3章 传输层



3.1 传输层服务

3.2 多路复用与分解

3.3 无连接传输: UDP

3.4 可靠数据传输原理

3.5 面向连接的传输: TCP

- 报文段结构
- 可靠数据传输
- 流量控制
- TCP连接管理

3.6 拥塞控制原理

3.7 TCP 拥塞控制

3-83

拥塞控制原理

拥塞:

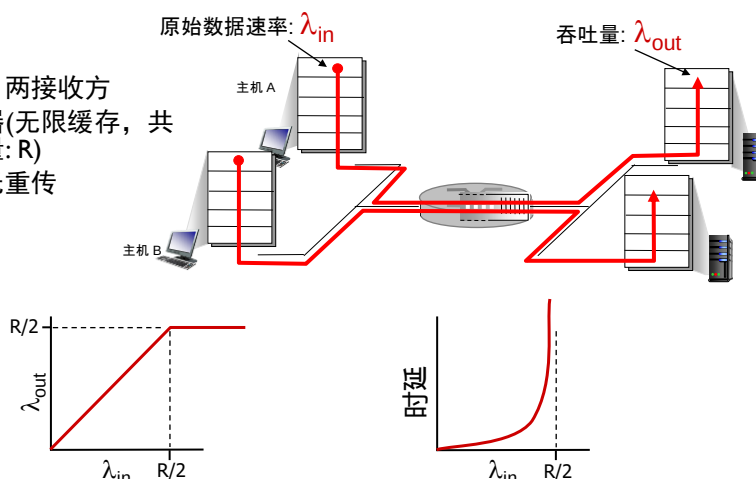
- 太多源主机发送太快太急，网络来不及处理
- 表现:
 - 长时延
 - 丢包
- 网络10大问题之一!

3-84

congestion causes and costs I

Assumptions:

- Two senders, two receivers
- One router (infinite buffer, shared link capacity: R)
- No packet loss, no retransmission

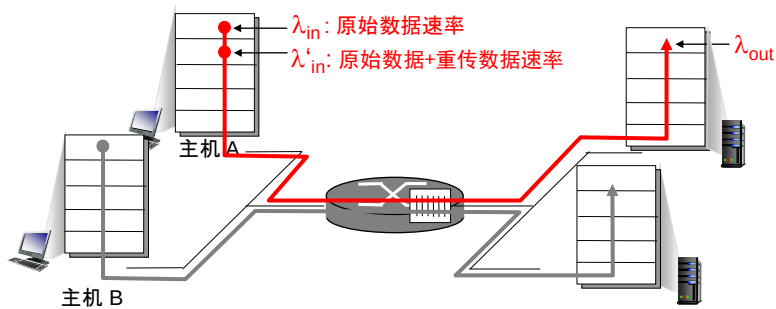


Congestion cost I: When the router input rate approaches link capacity, packet delay will be very large.

3-85

congestion causes and costs 2

- One router (finite buffer)
- Sender timeout retransmission
 - Application layer input = application layer output: $\lambda_{in} = \lambda_{out}$
 - Transport layer input includes retransmitted data: $\lambda'_{in} \geq \lambda_{in}$

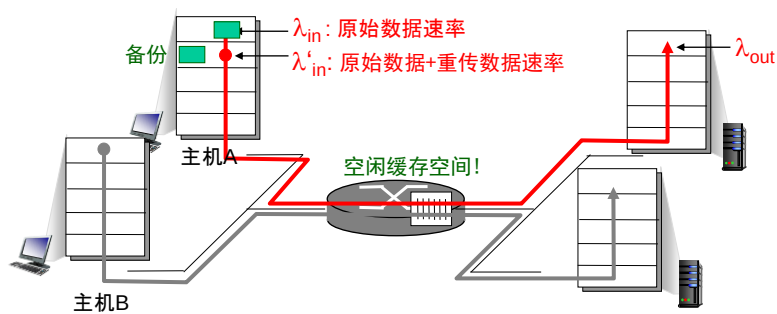
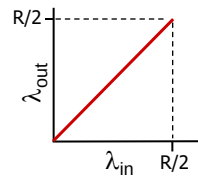


3-86

拥塞的原因与代价2

理想情况1:

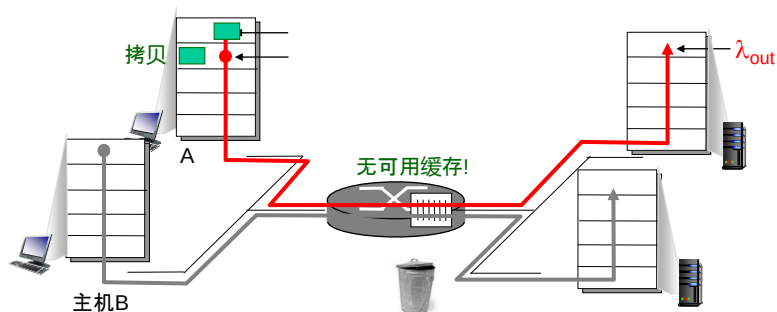
- 发送方只在路由器有空闲缓存时发送，无丢包



3-87

拥塞的原因与代价2

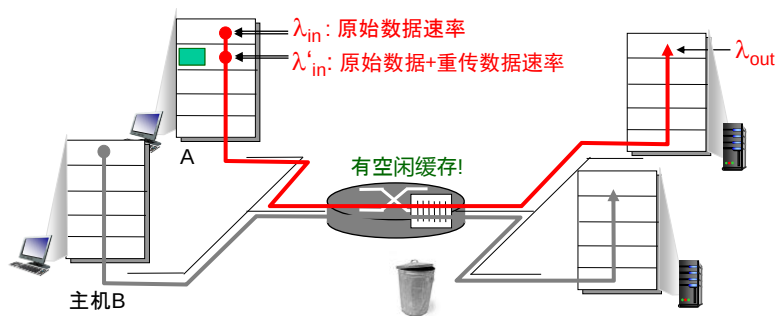
理想情况2: 由于路由器缓存满了，
丢包，发送方只对丢包进行重传



3-88

拥塞的原因与代价2

理想情况2：由于路由器缓存满了，
丢包，发送方只对丢包进行重传

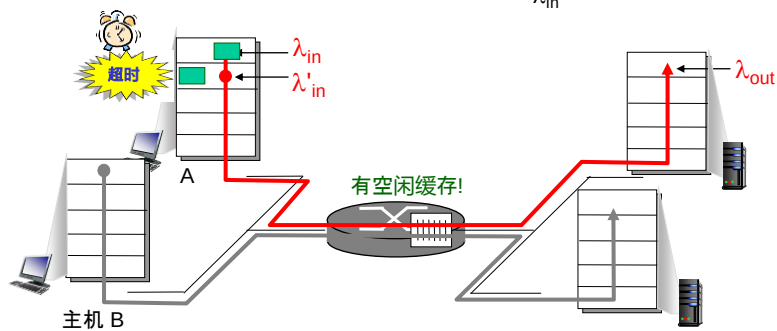
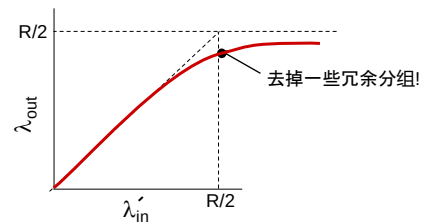


3-89

拥塞的原因与代价2

实际情况：

- 路由器缓存满了，丢包重传
- 发送方过早超时，发送冗余分组

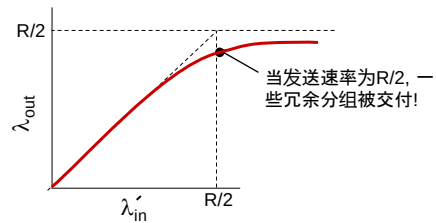


3-90

拥塞的原因与代价2

实际情况：

- 路由器缓存满了，丢包重传
- 发送方过早超时，发送冗余分组



拥塞代价二：路由器缓存溢出导致丢包，引发重传；
时延大导致过早超时，引发不必要重传

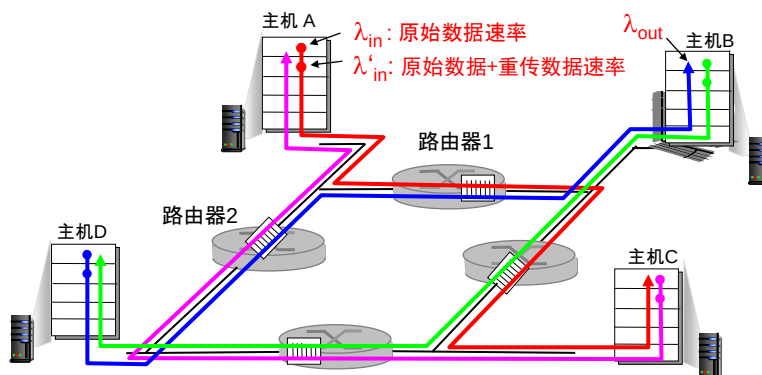
3-91

拥塞的原因与代价3

- 四个发送方
- 多跳路径
- 超时/重传

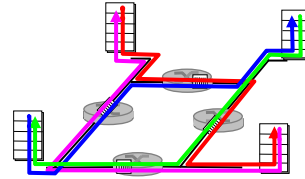
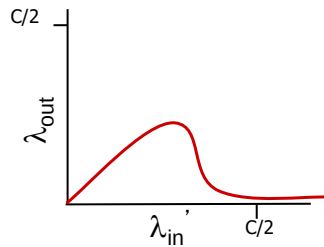
Q: 如果所有 λ_{in} 和 λ'_{in} 都增大，会怎样？

A: 当红色的 λ'_{in} 增大，所有到达路由器1的蓝色分组被丢弃，蓝色吞吐量为0，导致路由器2对蓝色分组的转发变成“无用功”



3-92

拥塞的原因与代价3



拥塞代价三：被丢弃的分组，其上游路由器的转发成为“无用功”

拥塞控制：控制发送方速度，使网络来得及转发

3-93

第3章 传输层



3.1 传输层服务

3.2 多路复用与分解

3.3 无连接传输: UDP

3.4 可靠数据传输原理

3.5 面向连接的传输: TCP

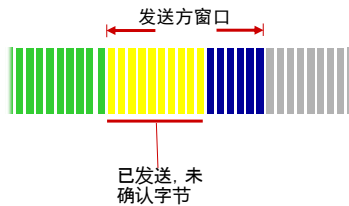
- 报文段结构
- 可靠数据传输
- 流量控制
- TCP连接管理

3.6 拥塞控制原理

3.7 TCP 拥塞控制

3-94

TCP拥塞控制



■ 发送方限制:

$$\text{发送方窗口} \leq \text{Min}\{\text{接收窗口, 拥塞窗口}\}$$

■ 接收窗口=接收方空闲缓存

■ 拥塞窗口随网络拥塞动态变化:

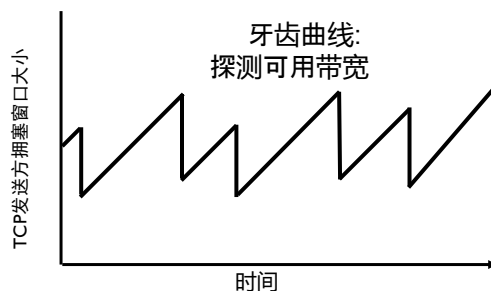
- 三个阶段: 慢启动、拥塞避免、快速恢复
- 丢包 (拥塞) 判断: 超时或收到3个冗余ACK

3-95

简单的TCP拥塞控制

■ 方法: 发送方增加发送速率 (增大窗口), 探测最大可用网络带宽, 直到丢包

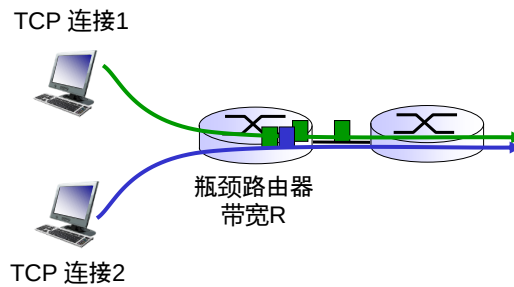
- **加法增:** 拥塞窗口每个RTT增加1个MSS, 直到发现丢包
- **乘法减:** 发现丢包后, 拥塞窗口减一半



3-96

TCP的公平性

如果K个TCP连接共享使用瓶颈带宽R，每个应获得 R/K

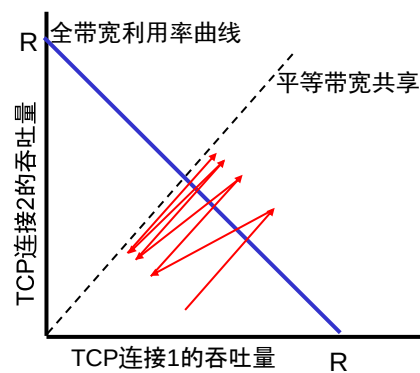


3-97

TCP的公平性

两个竞争的TCP连接：

- 拥塞窗口加法增时，斜率为1
- 窗口大小乘法减时，各减一半



3-98

TCP的公平性

UDP

- 多媒体传输通常不使用TCP
 - 拥塞控制会限制流量
- 使用UDP:
 - 以恒定速率发送音视频、容忍部分丢包

公平、并行的TCP连接

- HTTP使用，各连接公平分享瓶颈带宽